

PATTERN PRINTING IN JAVA

1. What is Pattern Printing?

Pattern printing is a programming technique in which we use **loops and conditional statements** to print characters, numbers, or symbols in a specific arrangement.

For example:

```
*  
**  
***  
****  
*****
```

or:

```
1  
12  
123  
1234  
12345
```

Although these problems look simple, they are extremely useful for learning:

- Nested loops
- `for` loops
- `while` loops
- Row and column relationships
- Space management
- Mathematical observations
- Conditional logic
- Boundary conditions
- Dry running code

These concepts are frequently useful in DSA problems.

2. Why Should We Learn Pattern Printing?

Pattern problems help us develop the ability to convert a visual pattern into an algorithm.

For example:

```
*  
**  
***  
****
```

We need to observe:

Row	Number of Stars
1	1
2	2
3	3
4	4

So we identify:

Number of stars = row number

That observation becomes our loop condition.

Pattern printing teaches us to ask:

"What changes from one row to the next?"

This thinking is important in DSA.

3. Basic Structure of Pattern Printing

Most pattern problems can be divided into:

Rows

↓

Columns

↓

What should be printed?

The most common structure is:

```
for (int i = 1; i <= n; i++) {  
  
    for (int j = 1; j <= n; j++) {  
  
        // print something  
  
    }  
  
    System.out.println();  
}
```

Here:

- Outer loop → controls rows
 - Inner loop → controls columns
 - `System.out.println()` → moves to next row
-

4. Understanding `print()` vs `println()`

`System.out.print()`

Prints without moving to the next line.

```
System.out.print("**");  
System.out.print("**");
```

Output:

```
**
```

`System.out.println()`

Prints and moves to the next line.

```
System.out.println("**");  
System.out.println("**");
```

Output:

```
*  
*
```

Important Rule

Inside the inner loop:

```
System.out.print();
```

After the inner loop:

```
System.out.println();
```

5. The Golden Rule of Pattern Printing

Before writing code:

Step 1

Count the number of rows.

Step 2

For every row, determine:

- How many spaces?
- How many stars?
- How many numbers?
- What value should be printed?

Step 3

Find the mathematical relationship.

6. Pattern 1 — Solid Square

Problem

Print:

```
*****
*****
*****
*****
*****
```

For $n = 5$.

Observation

There are:

- 5 rows
- 5 stars in every row

Therefore:

Rows = n

Columns = n

Java Code

```
import java.util.*;

class Main {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= n; j++) {
                System.out.print("*");
            }

            System.out.println();
        }
    }
}
```

Output

```
*****
*****
*****
*****
*****
```

Complexity

Time:

$O(n^2)$

Space:

$O(1)$

7. Pattern 2 — Increasing Right Triangle

Output

```
*  
**  
***  
****  
*****
```

Observation

Row	Stars
-----	-------

1	1
2	2
3	3
4	4
5	5

Therefore:

Stars = row number

Java Code

```
class Main {  
    public static void main(String[] args) {  
  
        int n = 5;  
  
        for (int i = 1; i <= n; i++) {  
  
            for (int j = 1; j <= i; j++) {  
                System.out.print("*");  
            }  
        }  
    }  
}
```

```
        System.out.println();
    }
}
}
```

8. Pattern 3 — Decreasing Right Triangle

Output

```
*****
****
***
**
*
```

Observation

Row	Stars
-----	-------

1	5
---	---

2	4
---	---

3	3
---	---

4	2
---	---

5	1
---	---

Formula:

Stars = $n - i + 1$

Java Code

```
class Main {
    public static void main(String[] args) {

        int n = 5;
```

```
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= n - i + 1; j++) {  
            System.out.print("*");  
        }  
        System.out.println();  
    }  
}
```

9. Pattern 4 — Increasing Number Triangle

Output

```
1  
12  
123  
1234  
12345
```

Observation

For row **i**:

Print numbers from 1 to i

Java Code

```
class Main {  
    public static void main(String[] args) {  
        int n = 5;  
        for (int i = 1; i <= n; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print(j);  
            }  
        }  
    }  
}
```

```
    }  
    System.out.println();  
  }  
}  

```

10. Pattern 5 — Same Number in Each Row

Output

```
1  
22  
333  
4444  
55555
```

Observation

For every row:

Print the row number i

Java Code

```
class Main {  
    public static void main(String[] args) {  
  
        int n = 5;  
  
        for (int i = 1; i <= n; i++) {  
  
            for (int j = 1; j <= i; j++) {  
                System.out.print(i);  
            }  
  
            System.out.println();  
        }  
    }  
}
```

```
}  
}
```

11. Pattern 6 — Continuous Numbers

Output

```
1  
23  
456  
789
```

Observation

Numbers do not restart from 1.

We need a separate variable:

```
int num = 1;
```

Every time we print a number:

```
num++;
```

Java Code

```
class Main {  
    public static void main(String[] args) {  
  
        int n = 4;  
        int num = 1;  
  
        for (int i = 1; i <= n; i++) {  
  
            for (int j = 1; j <= i; j++) {  
                System.out.print(num);  
                num++;  
            }  
        }  
    }  
}
```

```
        System.out.println();
    }
}
}
```

12. Pattern 7 — Reverse Number Triangle

Output

```
12345
1234
123
12
1
```

Observation

Number of elements:

$n - i + 1$

But numbers always start from 1.

Java Code

```
class Main {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= n - i + 1; j++) {
                System.out.print(j);
            }

            System.out.println(); } } }
```

13. Pattern 8 — Right-Aligned Triangle

Output

```
*
**
***
****
*****
```

This is where **spaces** become important.

Observation

For every row:

Spaces + Stars = n

For $n = 5$:

Row	Space s	Stars
1	4	1
2	3	2
3	2	3
4	1	4
5	0	5

Therefore:

Spaces = $n - i$

Stars = i

Java Code

```
class Main {
    public static void main(String[] args) {
```

```
int n = 5;

for (int i = 1; i <= n; i++) {

    // Spaces
    for (int j = 1; j <= n - i; j++) {
        System.out.print(" ");
    }

    // Stars
    for (int j = 1; j <= i; j++) {
        System.out.print("*");
    }

    System.out.println();
}
}
```

14. Pattern 9 — Pyramid

Output

```
  *
 ***
*****
*****
*****
```

This is one of the most important patterns.

Observation

For every row:

Spaces = $n - i$
Stars = $2 * i - 1$

For $n = 5$:

Row	Space s	Stars
1	4	1
2	3	3
3	2	5
4	1	7
5	0	9

Java Code

```
class Main {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            // Spaces
            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }

            // Stars
            for (int j = 1; j <= 2 * i - 1; j++) {
                System.out.print("*");
            }

            System.out.println();
        }
    }
}
```

15. Pattern 10 — Inverted Pyramid

Output

```
*****
*****
*****
***
*
```

Formula

Spaces = i - 1
Stars = 2 * (n - i) + 1

Java Code

```
class Main {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            // Spaces
            for (int j = 1; j < i; j++) {
                System.out.print(" ");
            }

            // Stars
            for (int j = 1; j <= 2 * (n - i) + 1; j++) {
                System.out.print("*");
            }

            System.out.println();
        }
    }
}
```

16. Pattern 11 — Diamond

Output

```
*
```

```
***
****
*****
*****
*****
*****
***
*
```

A diamond can be divided into:

Upper Pyramid

+

Lower Inverted Pyramid

Java Code

```
class Main {
    public static void main(String[] args) {

        int n = 5;

        // Upper half
        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }

            for (int j = 1; j <= 2 * i - 1; j++) {
                System.out.print("*");
            }

            System.out.println();
        }

        // Lower half
        for (int i = n - 1; i >= 1; i--) {

            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }

            for (int j = 1; j <= 2 * i - 1; j++) {
                System.out.print("*");
            }
        }
    }
}
```

```
    }  
    System.out.println();  
  }  
}  

```

17. Pattern 12 — Hollow Square

Output

```
* * * * *  
*       *  
*       *  
*       *  
* * * * *
```

This introduces an important concept:

Boundary conditions

Observation

Print * when:

First row

OR

Last row

OR

First column

OR

Last column

Otherwise print space.

Condition

if (i == 1 || i == n || j == 1 || j == n)

Java Code

```
class Main {  
    public static void main(String[] args) {  
  
        int n = 5;  
  
        for (int i = 1; i <= n; i++) {  
  
            for (int j = 1; j <= n; j++) {  
  
                if (i == 1 || i == n || j == 1 || j == n) {  
                    System.out.print("*");  
                } else {  
                    System.out.print(" ");  
                }  
            }  
  
            System.out.println();  
        }  
    }  
}
```

18. Pattern 13 — Hollow Rectangle

For:

n = 4
m = 7

Output:

```
* * * * *
*           *
*           *
* * * * *

```

Condition

i == 1 || i == n || j == 1 || j == m

Java Code

```
class Main {
    public static void main(String[] args) {

        int n = 4;
        int m = 7;

        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= m; j++) {

                if (i == 1 || i == n || j == 1 || j == m) {
                    System.out.print("*");
                } else {
                    System.out.print(" ");
                }
            }

            System.out.println();
        }
    }
}
```

19. Pattern 14 — Hollow Pyramid

Output

```
  *
 * *
*   *
*     *
*****
```

Important Observation

For each row:

- First star → beginning
- Last star → ending
- Last row → completely filled

Condition:

```
j == 1
OR
j == 2*i - 1
OR
i == n
```

20. Pattern 15 — Floyd's Triangle

Output

```
1
23
456
78910
```

Java Code

```
class Main {
    public static void main(String[] args) {

        int n = 4;
        int num = 1;
```

```
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= i; j++) {  
            System.out.print(num + " ");  
            num++;  
        }  
        System.out.println();  
    }  
}
```

21. Pattern 16 — 0-1 Triangle

Output

```
1  
01  
101  
0101  
10101
```

Observation

We need alternating values.

One simple condition is:

If $(i + j)$ is even $\rightarrow 1$
Else $\rightarrow 0$

Java Code

```
class Main {  
    public static void main(String[] args) {  
        int n = 5;  
        for (int i = 1; i <= n; i++) {
```

```

        for (int j = 1; j <= i; j++) {

            if ((i + j) % 2 == 0) {
                System.out.print("1");
            } else {
                System.out.print("0");
            }
        }

        System.out.println();
    }
}

```

22. Pattern 17 — Number Pyramid

Output

```

1
123
12345
1234567
123456789

```

Formula

Spaces = $n - i$
 Numbers = $2*i - 1$

Java Code

```

class Main {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }
        }
    }
}

```

```

    }

    for (int j = 1; j <= 2 * i - 1; j++) {
        System.out.print(j);
    }

    System.out.println();
}
}
}

```

23. Pattern 18 — Palindromic Number Pyramid

Output

```

1
121
12321
1234321
123454321

```

This is a very useful placement-level pattern.

Observation

Each row has two parts:

Increasing numbers

+

Decreasing numbers

For row 4:

1234 + 321

Java Code

```

class Main {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 1; i <= n; i++) {

            // Spaces
            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }

            // Increasing
            for (int j = 1; j <= i; j++) {
                System.out.print(j);
            }

            // Decreasing
            for (int j = i - 1; j >= 1; j--) {
                System.out.print(j);
            }

            System.out.println();
        }
    }
}

```

24. Pattern 19 — Butterfly Pattern

Output

```

*      *
**    **

```

```
***      ***
****     ****
*****
****     ****
***      ***
**       **
*        *
```

This is an excellent problem for understanding multiple components in a row.

Each row has:

Left stars
+
Middle spaces
+
Right stars

For the upper half:

Stars = i
Spaces = 2 * (n - i)

Java Code

```
class Main {
    public static void main(String[] args) {

        int n = 5;

        // Upper half
        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= i; j++) {
                System.out.print("*");
            }

            for (int j = 1; j <= 2 * (n - i); j++) {
                System.out.print(" ");
            }

            for (int j = 1; j <= i; j++) {
```

```

        System.out.print("**");
    }

    System.out.println();
}

// Lower half
for (int i = n; i >= 1; i--) {

    for (int j = 1; j <= i; j++) {
        System.out.print("**");
    }

    for (int j = 1; j <= 2 * (n - i); j++) {
        System.out.print(" ");
    }

    for (int j = 1; j <= i; j++) {
        System.out.print("**");
    }

    System.out.println();
}
}
}

```

Note

If you don't want the middle row to be printed twice, use:

```
for (int i = n - 1; i >= 1; i--)
```

for the second half.

25. Pattern 20 — X Pattern

Output

```

*  *
*  *
*
*  *
*  *

```

This is an important **condition-based pattern**.

Observation

Print * when:

$i == j$

OR

$i + j == n + 1$

Java Code

```
class Main {  
    public static void main(String[] args) {  
  
        int n = 5;  
  
        for (int i = 1; i <= n; i++) {  
  
            for (int j = 1; j <= n; j++) {  
  
                if (i == j || i + j == n + 1) {  
                    System.out.print("*");  
                } else {  
                    System.out.print(" ");  
                }  
            }  
        }  
  
        System.out.println();  
    }  
}
```

26. Pattern 21 — Plus (+) Pattern

Output

```

      *
      *
*****
      *
      *

```

Observation

Print * when:

i == middle row

OR

j == middle column

For odd n :

$$i == n / 2 + 1$$

or zero-indexed:

$$i == n / 2$$

27. Pattern 22 — Hollow Diamond

Output

```

      *
    * *
  *   *
*     *
*     *
*     *
*     *
*     *
*     *
*     *
  *   *
    * *
      *

```

The important idea is:

First understand the filled diamond, then identify which positions form its boundary.

This is a good example of moving from **shape-based thinking** to **condition-based thinking**.

28. Pattern 23 — Pascal's Triangle

Output

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Pascal's Triangle is slightly different because values depend on combinations.

The mathematical formula is:

$$C(n, r) = n! / (r! * (n-r)!)$$

But calculating factorial repeatedly is inefficient.

A better approach is to calculate each value from the previous value.

Java Code

```
class Main {
    public static void main(String[] args) {

        int n = 5;

        for (int i = 0; i < n; i++) {

            // Spaces
            for (int j = 0; j < n - i - 1; j++) {
                System.out.print(" ");
            }
        }
    }
}
```

```

        long value = 1;

        for (int j = 0; j <= i; j++) {

            System.out.print(value + " ");

            value = value * (i - j) / (j + 1);
        }

        System.out.println();
    }
}

```

29. The Most Important Pattern-Printing Technique

Instead of memorizing patterns, learn this framework:

For every row:

1. How many spaces?
2. How many elements?
3. What should each element contain?

For example:

```

***
****
*****

```

For each row:

Spaces = $n - i$
 Stars = $2*i - 1$

Therefore:

```

for (int i = 1; i <= n; i++) {

    // spaces

```

```

    for (int j = 1; j <= n - i; j++) {
        System.out.print(" ");
    }

    // stars
    for (int j = 1; j <= 2 * i - 1; j++) {
        System.out.print("*");
    }

    System.out.println();
}

```

30. Pattern Classification

Almost every pattern can be classified into one of these categories:

Category 1 — Fixed Size

Example:

```

*****
*****
*****
*****
*****

```

Relationship:

columns = n

Category 2 — Increasing

Example:

```

*
**
***
****

```

Relationship:

columns = i

Category 3 — Decreasing

Example:

```
****
***
**
*
```

Relationship:

columns = n-i+1

Category 4 — Space + Elements

Example:

```
 *
**
***
****
```

Relationship:

spaces = n-i
elements = i

Category 5 — Odd Number Pattern

Example:

```
 *
***
*****
*****
```

Relationship:

elements = $2*i-1$

Category 6 — Boundary Pattern

Example:

```
*****
*  *
*  *
*****
```

Uses conditions such as:

```
i == 1
i == n
j == 1
j == n
```

Category 7 — Diagonal Pattern

Example:

```
*  *
*  *
*
*  *
*  *
```

Uses:

$i == j$

and:

$i + j == n + 1$

31. How to Solve Any Pattern

Use the following 5-step strategy.

Step 1 — Count Rows

Ask:

How many rows are there?

Usually:

```
for (int i = 1; i <= n; i++)
```

Step 2 — Analyze One Row

Take:

```
*****
```

Ask:

How many elements?

Answer:

5

Step 3 — Create a Row Table

For example:

```
*  
***  
*****  
*****
```

Create:

Row	Space s	Stars
-----	------------	-------

1	3	1
2	2	3
3	1	5
4	0	7

Now identify:

Spaces = $n-i$

Stars = $2*i-1$

Step 4 — Convert Relationships into Loops

for (int j = 1; j <= n - i; j++)

and:

for (int j = 1; j <= 2 * i - 1; j++)

Step 5 — Dry Run

Always test with:

$n = 3$

before testing:

$n = 10$

Small inputs make mistakes easier to identify.

32. Common Mistakes

Mistake 1 — Forgetting `println()`

Wrong:

```
for (...) {  
    for (...) {  
        System.out.print("");  
    }  
}
```

Everything appears on one line.

Correct:

```
for (...) {  
    for (...) {  
        System.out.print("");  
    }  
  
    System.out.println();  
}
```

33. Common Mistake — Wrong Loop Boundary

For example:

$j < i$

and:

$j \leq i$

are different.

If:

$i = 3$

then:

`j <= i`

prints 3 elements.

But:

`j < i`

prints only 2 elements.

34. Common Mistake — Incorrect Spaces

For:

```
*  
**  
***  
****  
*****
```

Students often focus only on stars.

But the pattern consists of:

Spaces + Stars

Always analyze both.

35. Common Mistake — Starting From 0 vs 1

Both are valid.

1-based

```
for (int i = 1; i <= n; i++)
```

0-based

```
for (int i = 0; i < n; i++)
```

For beginners, 1-based indexing is often easier for mathematical pattern formulas.

36. Common Mistake — Overcomplicating the Problem

Students sometimes use:

```
if  
else if  
else if  
else if
```

for every pattern.

Instead, first ask:

"Can I divide each row into independent sections?"

For example:

spaces | stars | spaces | stars

Then write one loop for each section.

37. Pattern Printing and Nested Loops

Suppose:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
          
    }  
}
```

The inner loop executes:

n times

for every outer-loop iteration.

Therefore:

$$n \times n = n^2$$

Time complexity:

$$O(n^2)$$

38. Complexity of Pattern Problems

Most basic pattern problems have:

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Why?

Because we generally use:

nested loops

and only a few variables.

Important

If the output itself contains $O(n^2)$ characters, we cannot generally print it in less than $O(n^2)$ time because simply producing the output takes that much work.

39. Important Formulas

These formulas should be remembered.

Increasing Triangle

elements = i

Decreasing Triangle

elements = $n-i+1$

Right-Aligned Triangle

spaces = $n-i$

elements = i

Pyramid

spaces = $n-i$

elements = $2*i-1$

Inverted Pyramid

spaces = $i-1$

elements = $2*(n-i)+1$

40. Quick Pattern Formula Sheet

Pattern	Space s	Elements
Increasing Triangle	0	i
Decreasing Triangle	0	$n-i+1$
Right Triangle	$n-i$	i
Pyramid	$n-i$	$2*i-1$
Inverted Pyramid	$i-1$	$2*(n-i)+1$

Keep this table handy while solving patterns.
